



AI は数当てゲームの「うまい予想」を どうやって覚えるのか — hit & blow

1 / 2

数当てゲーム Hit & Blow の CPU を、強化学習(深層 Q 学習 / DQN)で作りました。CPU には「どう予想すればよいか」を教えていません。数万回の対局で、早く当てるほど高い点数(報酬)がもらえるというルールだけから、戦略を自分で身につけます。

01 Hit & Blow のルール

相手が決めた 0~9 の重複しない 4 つの数字を当てるゲームです。予想を言うたびに、次の 2 種類のヒントが返ってきます。

● Hit
数字も位置も合っている個数

○ Blow
数字は合っているが位置が違う個数

秘密の数字
相手には見えない

7683

予想 1

0123

1 Hit 0 Blow

予想 2

3670

1 Hit 2 Blow

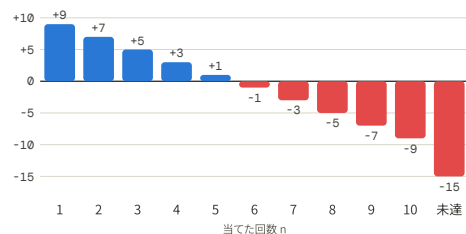
例: 秘密の数字が 7683 のとき。展示中のアプリでは、プレイヤーと CPU が同じ秘密の数字を交互に予想し、少ない回数で当てた方の勝ちです。

数字の組み合わせは全部で $10 \times 9 \times 8 \times 7 = 5,040$ 通り。慣れた人でも平均 5~6 回ほどかかります。

04 報酬の設計

AI は「報酬の合計」を大きくしようとします。そこで、早く当てるほど合計が高くなるように点数を決めました。

- 予想するたびに -1 点
- n 回目で当てたら $11 - n$ 点のボーナス
- 10 回で当てられなければ -5 点

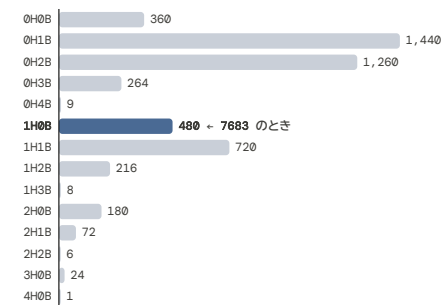


当てた回数 n ごとの報酬の合計 $(11 - 2n)$ 。1 回目なら +9、5 回目なら +1、当てられなければ -15。

報酬の決め方ひとつで AI の目標が変わります。もし「当てたら +1」だけにすると、何回かかっても同じ点数なので、早く当てようとしなくなってしまいます。

02 候補を絞り込むゲーム

ここまでの判定と矛盾しない秘密の数字をすべて集めたものを候補集合と呼びます。最初は 5,040 個。予想すると、返ってくる判定(14 種類)ごとに候補がグループに分かれ、そのうち 1 グループだけが残ります。



最初に 0123 と予想したときの、判定ごとの残り候補数。秘密が 7683 なら判定は 1 Hit 0 Blow で、480 個に絞られる。

良い予想とは、どの判定が返ってきても残りの候補が少なくなる予想。候補を「なるべく均等に、細かく」分ける予想です。

ただし本当に最善の予想を求めるには、何手も先まで全パターンを調べる必要があり、とても大変です。そこで今回は、AI に経験から学ばせました。

05 Q 値 — 手の将来の価値

状態 s で予想 a をしたあと、ゲームが終わるまでにもらえる報酬の合計の見込みを Q 値 $Q(s, a)$ と呼びます。Q 値が正しくわかれば、Q 値が最大の予想を選ぶだけで最善の戦略になります。

$$Q(s, a) = r + \gamma \cdot \max_{a'} Q(s', a')$$

ベルマン方程式: 今の価値 = すぐもらえる報酬 + 次の状態 s' で最善を尽くしたときの価値 (γ は将来の報酬の割引率。今回は 1)

学習済み AI(Hard)が見積もった Q 値

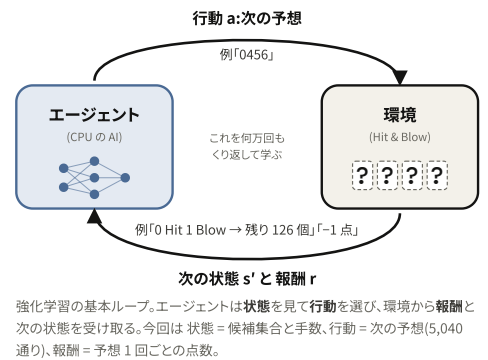
1 回目 0123 → 1 Hit 0 Blow(残り 480 個)の場面で、2 回目の予想を比べると:

予想	候補に含まれる?	最大のグループ	Q 値
0456	選んだ手	126	1.71
4713	×	132	1.63
4567	×	168	1.48
0124	×	180	1.10
0132	×	240	0.26

$Q \approx 1.7$ は「合計 5 回前後で当てられそう」という見積もり(2 回目以降の報酬の合計は $12 - 2n$ なので $n \approx 5.1$)。前の判定と矛盾する 0132 は候補を 2 グループに分けられず、Q 値が低い。

03 強化学習とは?

強化学習は、試行錯誤を通じて「どう行動すれば、もらえる報酬の合計が最大になるか」を学ぶ機械学習の方法です。囲碁の AlphaGo、ロボットの歩行制御、ゲーム AI などに使われています。



強化学習の基本ループ。エージェントは状態を見て行動を選び、環境から報酬と次の状態を受け取る。今回は 状態 = 候補集合と手数、行動 = 次の予想(5,040 通り)、報酬 = 予想 1 回ごとの点数。

	教師あり学習	強化学習(今回)
教えるもの	問題と正解のペア 例: 写真と「ねこ」	行動の結果のよしあし(報酬)だけ。正解の手は教えない
難しさ	正解データを大量に用意する必要がある	報酬が遅れて届く。どの手が良かったかを自分で見つける

06 学習の流れ

Q 値を表にして覚えようとしても、状態(候補集合)の種類が多すぎて表が作れません。そこで Q 値を予測するニューラルネットワークを使います。これが DQN(Deep Q-Network)です。



予測した Q 値がベルマン方程式を満たすように、ネットワークを少しずつ修正していきます。かわいい仕組みと結果は 2 枚目へ →



ニューラルネットワークで「手の価値」を予測する



AI と対戦してみよう

スマートフォンのブラウザで遊べます
www.chrom.jp/hit-and-blow

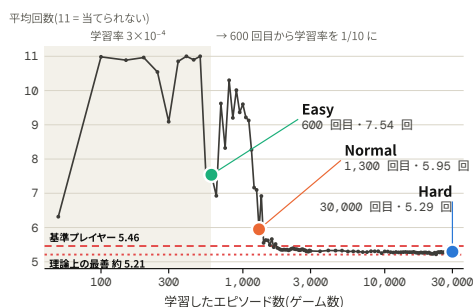
07 局面を数値で表す

ニューラルネットワークに渡せるのは数値だけです。そこで、状態と予想をそれぞれ数値の並び(特徴量)に変換します。

種類	内容	個数
状態	残り候補数(対数で0~1 に)	1
	何手目か / 残り 1 個か	2
	候補の中で、各桁に各数字が出てくる割合(4 桁 × 10)	40
予想ごと	候補に含まれるか / 一発で当たる確率	2
	残り候補数の期待値 / 最大グループの割合	2
	判定の分かれ方のエントロピー(情報量) / 判定の種類数	2
	各桁の数字の出やすさ / 残り候補数の対数の期待値	2

「候補をどれだけ細かく分けるか」を表す量は渡していますが、**どれをどう重視すればよいか**は教えていません。それを見つけるのが学習の仕事です。

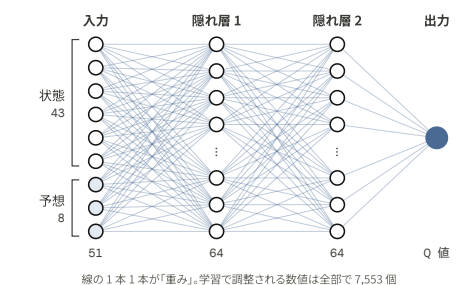
10 学習の進み方



学習の途中で保存したネットワークごとに、500 個の秘密の数字を当てさせたときの平均回数(当てられなければ 11 回と数える)。横軸は対数目盛。下に行くほど強い。

1 エピソード = 秘密の数字 1 つを当てるまでの 1 ゲーム。30,000 エピソード(ノートパソコンの CPU で約 15 分)学習しました。序盤は経験が少なく Q 値の予測が不安定なため、グラフが大きく上下します。

08 ネットワークの構造



線の 1 本 1 本が「重み」。学習で調整される数値は全部で 7,553 個
状態 43 + 予想 11 = 54 個の数値を入れると Q 値を 1 つ出す全結合ネットワーク(活性化関数 ReLU)。5,040 通りの予想すべてに同じネットワークを使い、Q 値が最大の予想を選ぶ。

	A: 出力 5,040 個	B: 出力 1 個(採用)
重みの数	約 336,000	7,553
学び方	5,040 個の値をばらばらに覚える	「よく分ける予想」という性質で覚えるので、他の予想にも応用が効く

重み(約 30 KB)はアプリに同梱し、ブラウザ上で計算しています。

11 3 つの難易度

学習途中のネットワークを、そのまま CPU の難易度にしてます。下は同じ秘密 7683 を 3 つの CPU に解かせた記録です(右の数字は残り候補数)。

	Easy	Normal	Hard
1	0123 ● 480	0123 ● 480	0123 ● 480
2	0132 ○ 240	0456 ○ 126	0456 ○ 126
3	0213 ● 120	4178 ○ ○ 20	4713 ● ○ 22
4	0124 — 60	7924 ● 3	6378 ○ ○ ○ ○ 2
5	5673 ● ● ○ 12	7583 ● ● ● 1	7683 正解
6	5783 ● ● ○ 2	7683 正解	
7	5863 ● ● ○ 1		
8	7683 正解		

- Easy(600 回目): 矛盾 前の判定からありえない予想をくり返し、候補を半分ずつしか減らせない。
- Normal(1,300 回目): 矛盾しない予想はできるが、絞り込みはまだ下手。
- Hard(30,000 回目): 候補外 3・4 回目は当たる可能性のない予想だが、22 → 2 個と大きく絞り込むために、あえて選んでいる。

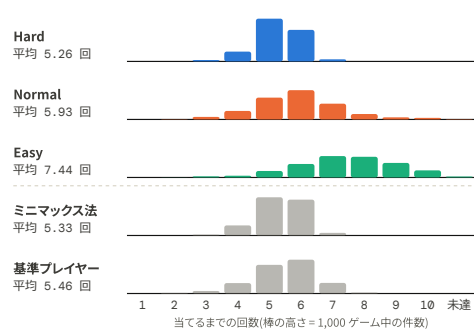
「当てにいく」より「情報を集める」方が得な場面があることを、AI は誰にも教わずに見つけました。

09 DQN の 4 つの工夫

自分の予測を使って自分を修正するため、Q 値が暴れて学習が崩れやすいことが知られています。DQN では次の工夫を組み合わせます。

- ① 経験再生**
対局の経験(状態・予想・報酬・次の状態)を最大 50,000 件ためておき、ランダムに 64 件ずつ取り出して学習する。続けて起きた似た経験ばかりで偏って学ばないようにする。
- ② ターゲットネットワーク**
ベルマン方程式の右辺(目標値)は、少し前のネットワークのコピーで計算する。コピーは 1,000 回の更新ごとに作り直す。目標が動き続けて「追いかけてこ」になるのを防ぐ。
- ③ ϵ -greedy(探索と活用)**
確率 ϵ でランダムな予想を試し、それ以外は Q 値が最大の予想を選ぶ。 ϵ は 1 から 0.05 まで徐々に下げ、最初はいろいろ試し、だんだん学んだことを活かす。
- ④ Double DQN**
次の状態で最善手を選ぶのは学習中のネットワーク、その手を評価するのはターゲットと役割を分け、Q 値を高く見積もりすぎる偏りを減らす。

12 結果とまとめ



同じ 1,000 個の秘密の数字で計測。
ミニマックス法:最悪の場合の残り候補数が最小になる予想を計算で選ぶ。
基準プレイヤー:矛盾しない候補からランダムに選ぶ(上手な人の目安)。

まとめ

- 報酬だけを手がかりに、AI は「矛盾しない予想をする」→「候補をよく分ける予想をする」と段階的に上達した。
- Hard の平均は 5.26 回・最悪でも 7 回。このゲームで理論上の最善とされる平均約 5.21 回にかなり近い。計算で作ったミニマックス法(平均 5.33 回)も、平均ではわずかに上回った。
- 候補外の予想で情報を集めるという、人間がなかなか思いつかない手も自分で見つけた。
- 今後の課題:相手に合わせた強さの自動調整、桁数や重複りなどのルール変更への対応。